Computing & Robotics

Year 4

Cambridge Early Years

Student Manual

Computing & Robotics

Year 4 · Student Manual
The Workshop

A student book for Lower Primary, written for pupils of about eight to nine years old. It teaches the one idea the whole subject rests on: a computer follows a plan, and the plan is yours to write.

Five benches in one workshop: plan, code, sort, signal and machine. Rufus the fox and Hazel the rabbit work beside you, and Cog does exactly what he is told.

Computing & Robotics · Year 4
Student Manual. First edition, 2026.

Prime Books is the publishing imprint of Prime School, Portugal.
www.primeschool.pt

Text, design and illustration. Written and designed by the Prime Books editorial studio. Original watercolour illustration produced for Prime School. All illustrations © Prime School 2026.

Interior 8.5 × 11 in. British English throughout. Printed in Portugal on paper from responsibly managed forests.

Contents

This edition carries Unit 1, The Plan Bench. Units 2 to 5, the Code Bench, the Sorting Bench, the Signal Bench and the Machine Bench, follow in the complete book.

The bench before anyone arrives. A plan sheet held flat by two stones, a tin of pencils, a rule and a mug. Everything in this book starts here.

Welcome to the Workshop

A computer cannot think.

That sounds like an insult, but it is the most useful thing you will learn all year, because it tells you exactly what your job is.

A computer is a machine that follows a plan. It does what the plan says, in the order the plan says it, and it does not stop to wonder whether that was what you meant. If the plan is good, the machine looks clever. If the plan has a gap in it, the machine walks straight into a bucket.

So the clever part is not the machine. The clever part is the plan, and the plan is yours.

In this book you work at five benches. At each one you make something a machine can follow. You start at the Plan Bench, where you learn to write steps so clear that nobody could get them wrong, and then you find out how easily somebody does, and how to hunt down the reason.

You will need paper more often than you need a screen. That is on purpose. The thinking happens on the paper first.

Five benches, one workshop. Rufus has the plan, Hazel is ready to test it, and Cog is waiting to be told what to do.

THE WORKSHOP · WHAT GETS MADE AT EACH BENCH

1. **The Plan Bench.** Steps in order. What an algorithm is, and what a bug is.
2. **The Code Bench.** Turning those steps into a script a computer runs.
3. **The Sorting Bench.** Collecting data, sorting it, and showing it so it can be read.
4. **The Signal Bench.** How machines are joined together, and how to stay safe on them.
5. **The Machine Bench.** What a computer is actually made of, inside and out.

This book is bench one. The other four follow.

You do not have to be good at computers to be good at this. You have to be willing to write down exactly what you mean, hand it over, and watch what happens. Everything else is practice.

Meet the workshop

Rufus

A fox who writes things down. He would rather spend ten minutes on a plan than one minute putting a mistake right. He is usually correct about this, and he knows it, which is a little annoying.

Hazel

A rabbit who tries things. She is the one who says “go on then, let us see”, and she is the reason the workshop finds out whether a plan actually works.

Cog

A small metal dog who does exactly what the plan says. Not what you meant. Not what would be sensible. Exactly what it says, every single time.

Four legs, a square head, two lenses for eyes and a stubby tail. No opinions at all.

Cog never guesses and never helps out. When something goes wrong in this book, it is never because Cog was being difficult. It is because the plan had a hole in it, and Cog walked through the hole without noticing, because that is what machines do.

COG CAN	COG CANNOT
take a number of paces	work out how far "over there" is
turn a quarter turn	notice that he is about to hit something
pick up a named object	choose between four objects
stop when told to stop	stop because stopping was obviously sensible
do the same thing a thousand times without getting bored	do the thing you meant instead of the thing you wrote

Everything in the right-hand column is your job. That is the deal you make with every machine you will ever program.

How to use this book

Every bench is built the same way, so once you know the shapes you will always know where you are.

SHAPE	WHAT IT MEANS
Open the workshop	A question to talk about before you start. There is no wrong answer yet.
A job to do	The real problem for this topic. Everything after it is you solving that.
At this bench you will	What you will be able to do by the end. Come back and check.
Warm up your thinking	Two minutes, no equipment, straight in.
Do you remember?	Something from earlier that you are about to need again.
Look and learn	The plan drawing and the new idea. Read the drawing first, then the words.
Your turn	Your work. Write, draw and tick in the book.
Did you know?	A true thing worth knowing that is not in the test.
Go a bit further	For when you have finished and want more.
Big challenge	Harder. Expect to get it wrong once.
Workshop show	The project at the end of the unit.
How did it go?	You judging your own work, honestly.
Now I can	The tick list. Only tick what is true.
Workshop words	Every new word, with its meaning.

The plan sheets

Wherever you see a dark sheet with a dashed line down its left edge, you are looking at a **plan**: the same kind of drawing Rufus keeps on the bench. Read the plan before you read the words around it.

The square codes

Some pages carry a small square code. It opens one page on the internet that is worth your time: a puzzle to solve, or a place to try something you have just read about. **An adult holds the device and opens the page.** There is nothing in this book you cannot finish without one.

Writing in this book

You are meant to. The pale boxes and the ruled lines are yours. Use a pencil, because you will want to change your mind, and changing your mind is most of what this subject is.

Getting set up

You do not need everything at once. Each bench tells you what to fetch.

Your workshop kit

- a soft pencil and a rubber, because plans get changed
- a small notebook that stays in your tray
- scissors (safety) and a glue stick
- a paper strip or a ruler for measuring steps
- ten wooden blocks, or ten of anything that stacks
- chalk, or a roll of masking tape, for marking a floor route
- a partner, which is the most important item on this list

Your kit, laid out. Nothing here needs a battery. The pencil does most of the work in Unit 1.

Workshop rules

1. Say the plan out loud before you write it. Your ears catch gaps your eyes miss.
2. One person reads, one person follows. Do not do both at once.
3. Follow the plan exactly, even when you can see it is wrong. **Especially** then.
4. When something goes wrong, say "found it", not "you got it wrong".
5. Walk with scissors closed and pointing down.
6. An adult holds the device for any square code.

> ✧ **DID YOU KNOW?**
>
> Rule 3 does the heavy lifting in this unit, and it is the one everybody breaks. Pupils want to repair a plan quietly as they follow it, which hides the very error the lesson is about. Following a broken plan all the way to the bucket is the lesson.

UNIT 1 · PLAN BENCH

1

Writing steps a machine can follow

An algorithm is a plan. A bug is a hole in the plan. Neither is the machine's fault.

At the Plan Bench you write down what to do, hand it to somebody who will not think for themselves, and find out what you forgot.

✓ **AT THIS BENCH YOU WILL**

- ✓ say what makes a list of steps an algorithm
- ✓ follow somebody else's algorithm exactly, even when it is wrong
- ✓ explain what an error is, and why it is not the machine's fault
- ✓ find an error in an algorithm on purpose, and put it right

Open the workshop

Think of something you can do without thinking: tying a lace, making toast, getting from your classroom to the school gate.

Now imagine explaining it to somebody who has never seen it done, cannot see what you are doing, and will not ask you a single question.

Where would they go wrong first?

UNIT 1 - TOPIC 1.1

What is an algorithm?

> ✦ WARM UP YOUR THINKING
>
> Stand up. In your head, count the steps from where you are standing to the classroom door. Now walk it and count for real.
>
> Were you right? Almost nobody is. Hold on to that feeling, because it is the whole of this unit: what we imagine is never quite what happens.

> ◆ A JOB TO DO
>
> *Rufus wants Cog to fetch the watering can from the end of the yard. He says, "Cog, fetch the watering can."*
>
> Cog does not move.
>
> Cog is not being awkward. Cog has no idea what "fetch" means, where "the end of the yard" is, or which of the four cans by the wall is a watering can. Rufus has given an instruction. He has not given a plan.

Look and learn

An **algorithm** is a list of steps that solves a problem, written so clearly that anyone, or anything, can follow it and get the same result.

Three things turn a list of steps into an algorithm.

IT MUST BE	WHICH MEANS
In order	Step 2 happens after step 1. Swap them and you get a different answer, or no answer at all.
Complete	Nothing is missing. There is no step you were "obviously" going to do.
Clear	Each step means one thing only. "Go a bit further" is not a step. "Take four paces" is.

Here is Rufus's second attempt, written properly.

PLAN 1.1 · FETCH THE WATERING CAN

1. Turn to face the stone wall.
2. Take eight paces forward.
3. Stop.
4. Pick up the green can with the long spout.
5. Take eight paces back.

Five steps. One action each. Nothing assumed.

Cog can do that. Every step is one action, nothing is left to be guessed, and the order is the order it happens in.

Now look at step 3. A person would never write "stop", because a person stops without being told. Cog does not. **If you do not write it, it does not happen.**

What it looks like when it is not an algorithm

Rufus's first go is worth keeping, because it shows the three faults at once.

PLAN 1.0 · THE FIRST GO, WHICH DID NOT WORK

1. Go over to the cans.
2. Get the watering can.

3 Come back.

Three lines. Not one of them is a step.

THE LINE	WHY COG CANNOT FOLLOW IT
Go over to the cans	How far is "over"? Which direction?
Get the watering can	Which of the four? What does "get" mean?
Come back	Back to where? Facing which way?

Every line makes sense to a person, because a person fills in the gaps without noticing. That is exactly why a person is a bad judge of their own plan, and why the next topic hands your plan to somebody else.

One line, one action. Rufus writes each step on its own line. It is far easier to find a mistake in a list than in a paragraph.

✧ **DID YOU KNOW?**

The word *algorithm* comes from a person's name. Al-Khwarizmi was a mathematician who worked in Baghdad about twelve hundred years ago and wrote books explaining how to work things out step by step. His name travelled into Latin, got bent out of shape on the way, and ended up as a word that now sits inside every phone on earth. He never saw a computer. He was writing for people.

● **YOUR TURN**

1. Here are four steps for making a jam sandwich, in the wrong order. Number them
1 to 4 in the boxes.

- ☐ Spread the jam on one slice.
- ☐ Take two slices of bread.
- ☐ Put the second slice on top.
- ☐ Open the jar.

2. One step is missing from this plan for going out to break. Write it in.

PLAN 1.2 · GOING OUT TO BREAK

1 Stand up.

2 Push your chair in.

3 ____________________

4 Walk to the door.

3 Write an algorithm to get from your seat to the classroom door. Four steps, one
action each. A partner must be able to follow it with their eyes shut, so no step may say "look".

YOUR PLAN · SEAT TO DOOR

1 ______

2 ______

3 ______

4 ______

→ GO A BIT FURTHER

Rewrite your door algorithm so that it works from **any** seat in the room, not just yours. What has to change?

Most people find they need a step that counts something rather than a step that names a place.

UNIT 1 - TOPIC 1.2

Following an algorithm

↺ DO YOU REMEMBER?

What are the three things that turn a list of steps into an algorithm?

In order, ______ ______

◆ A JOB TO DO

Hazel has Rufus's plan in her paw. Rufus is going to read it out, one line at a time, and Hazel is going to be the machine.

The rule is the hard bit: Hazel must do what the step says, not what she can see it ought to say.

Look and learn

Following an algorithm exactly is harder than writing one, because your brain keeps helping. It quietly closes gaps, fixes the order and adds the obvious step, and then you never find out that the plan was broken.

So we split the jobs up, the way a real workshop does.

ROLE	WHAT THEY DO	WHAT THEY MUST NOT DO
The writer	Reads each step aloud, once, in order	Explain, mime, or add "you know, the green one"
The machine	Does exactly what the step says	Guess, tidy up, or skip ahead
The watcher	Notes down where it went wrong	Shout it out while it is happening

Running a plan step by step and watching what happens at each one is called **tracing**. A watcher traces with a table like this.

STEP	WHAT IT SAYS	WHAT THE MACHINE DID
1	Turn to face the stone wall	Turned. Facing the wall.
2	Take eight paces forward	Walked into the bucket at pace six.
3	Stop	Never reached.

The table is doing something clever and simple at the same time. It puts what the plan *said* next to what actually *happened*, so the gap between them is impossible to miss.

■ **LOOK AND LEARN**

Hazel traces the door plan from 1.1 while Rufus reads it out. She is the watcher this time, so she writes and says nothing.

STEP	WHAT IT SAYS	WHAT THE MACHINE DID
1	Stand up	Stood up. Still behind the desk.
2	Take six paces forward	Two paces, then the desk.
3	Turn left	Never reached.

What Hazel wrote at the bottom: *step 1 was fine, step 2 was not. The plan never said to come out from behind the desk.*

Notice what she did not write. She did not write "the plan is rubbish", and she did not write a fix. Naming the last good step is the whole job of a watcher. Fixing comes later, in 1.4, and it comes one change at a time.

Being the machine. Hazel keeps her arms out and her eyes forward. It looks silly. It works, because it stops her helping.

YOUR TURN

1. Work in threes: a writer, a machine and a watcher. Use the door algorithm you wrote in 1.1. Run it once. Then swap roles and run it again.

2. Trace your own plan as you run it.

STEP	WHAT IT SAYS	WHAT THE MACHINE DID
1		
2		
3		
4		

3. Tick the one that is true for your plan.

- ☐ The machine ended up at the door.
- ☐ The machine ended up somewhere else.
- ☐ The machine stopped because a step made no sense.

▦ SQUARE CODE · KIDBOTS

Free classroom activities where one person is the robot and the others write the program. From CS Unplugged at the University of Canterbury. Ages 8 to 10, no computer needed, no account needed.

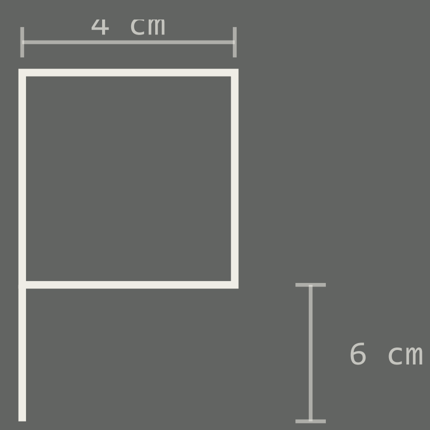
Why it is here. It gives your teacher fifty more routes to run when your class has used up the ones in this book.

AN ADULT OPENS THIS PAGE

▲ **BIG CHALLENGE**

Write an algorithm for drawing this shape, without showing it to your partner.

PLAN 1.3 · THE SHAPE



A 4 cm square, with a 6 cm line down from its bottom left corner.

Your partner draws only from your words. Then compare the two drawings.

WHAT YOU DESCRIBED	WHAT YOUR PARTNER DREW

Most pairs get this wrong the first time, and nearly always for the same reason: the writer said what to draw, but not **where to start** or **how big**.

Write the one step you would add if you ran it again.

UNIT 1 - TOPIC 1.3

What is an error?

✦ WARM UP YOUR THINKING

Say this instruction out loud: "Put the book on the table."

Now find three ways somebody could follow it correctly and still not do what you meant.

◆ A JOB TO DO

Cog is running Rufus's fetch plan. At step 2 he takes eight paces forward, straight into a tin bucket, and knocks it over.

Hazel says, "Cog's broken."

Rufus does not even look up. He says, "Cog isn't broken. My plan is."

▸ Look and learn

An **error**, or a **bug**, is a place where a plan does not do what its writer meant it to do.

Read that again, because the important word is *meant*. Cog followed step 2 perfectly. Eight paces forward is exactly what he did. The plan and the world disagreed, and the plan lost.

Errors are not naughtiness and they are not stupidity. They are the ordinary result of a person writing for something that does not think. Every programmer alive makes them every day, which is why finding them is a skill with a name and a method, rather than something to be embarrassed about.

There are three kinds you will meet all year.

KIND OF ERROR	WHAT WENT WRONG	IN THE FETCH PLAN
A missing step	Something you did without thinking was never written	Nobody wrote "move the bucket"
A step in the wrong place	Every step is there, but not in that order	Picking up the can before walking to it
A step that is not clear	The step can be read in more than one way	"Take the can" when four cans are standing there

PLAN 1.4 · THE SAME JOB, THREE WAYS TO BREAK IT

MISSING

```
1 walk to wall
2 pick up can
3 walk back

a 4th step is
missing here
```

ORDER

```
1 pick up can
2 walk to wall
3 walk back

these two are
swapped
```

NOT CLEAR

```
1 walk to wall
2 take the can
3 walk back

which can?
there are four
```

The arrow in each margin points at the line that is wrong. Left: a step that should be there is not. Middle: every step is present, in the wrong order. Right: one step can be read more than one way.

Cog is perfectly content. He is standing in a bucket, and he did exactly what he was told. Look at Rufus: he is already reading the plan, not the mess.

✧ **DID YOU KNOW?**

Computer people really do call errors *bugs*. In 1947, the team running a huge machine at Harvard University called the Mark II found a moth caught inside it. They taped the moth into their logbook and wrote beside it: "First actual case of bug being found." It was a joke. Engineers had been calling faults bugs since at least the 1870s, and Thomas Edison used the word in a letter in 1878, so the moth did not invent the word. It is just the only bug anybody kept.

● **YOUR TURN**

1. Each plan below has one error. Write which kind it is: **missing**, **order**, or **not clear**.

a. 1. Pour the milk. 2. Fetch a glass. Kind of error: ____________________

b. 1. Open the tin of paint. 2. Paint the fence. 3. Wash the brush. Kind of error:

____

c. 1. Walk to the wall. 2. Turn. 3. Walk back. Kind of error: ____________________

2. Look at plan (c) again. Write step 2 out properly, so that it can only be read one way.

____

3. Tick the one that is true.

☐ An error means the machine has gone wrong.

☐ An error means the plan has gone wrong.

4. Sort these three sentences into the right column. Write the letter in the box.

a "Draw a big circle." **b** "Put the lid on. Open the jar." **c** "Boil the kettle. Pour the water. Put in the teabag."

MISSING STEP	WRONG ORDER	NOT CLEAR

Sentence **c** is the interesting one. Every step is clear, and nothing is missing, but you will still be drinking hot water. Talk about why with a partner before you write anything.

UNIT 1 - TOPIC 1.4

Finding and fixing errors

↺ DO YOU REMEMBER?

Name the three kinds of error.

____________ ____________ ____________

◆ A JOB TO DO

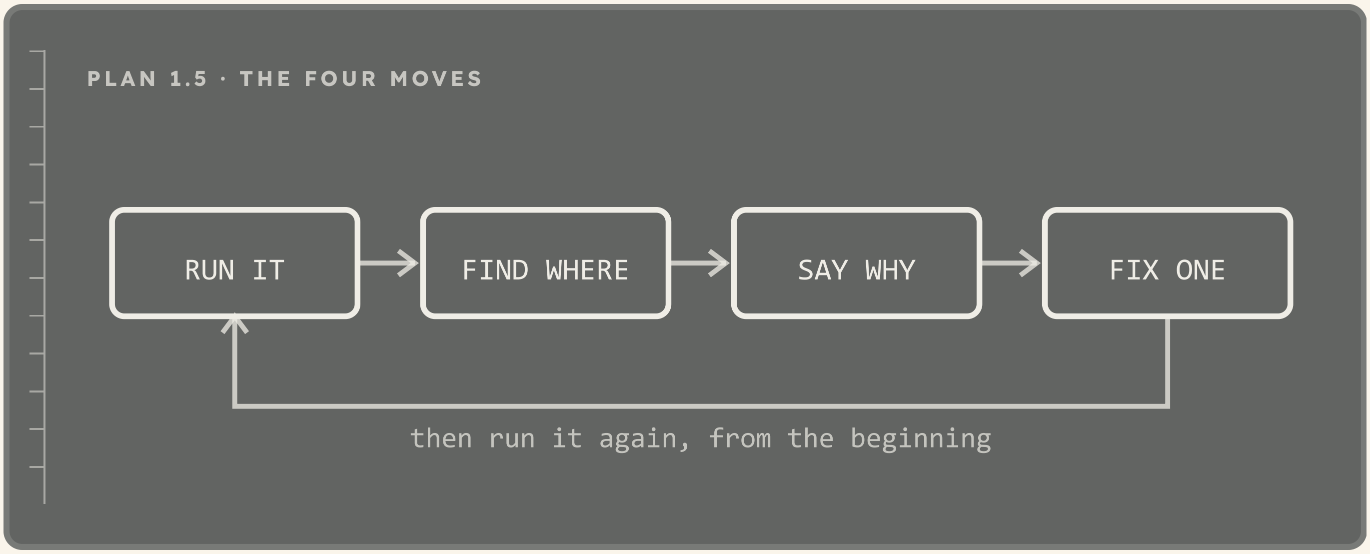
The plan is broken and nobody knows where.

Guessing is slow, and it is annoying: you change something, run it again, it still fails, and now you have two problems, because you have also changed something that was fine.

There is a better way, and it is not cleverness. It is method.

▸ Look and learn

Debugging is finding the error in a plan and putting it right. You do it in four moves, in this order, every time.



	MOVE	WHAT YOU ACTUALLY DO
1	**Run it**	Follow the plan exactly and let it fail. Do not repair it while you run it.
2	**Find where**	Note the last step that was still correct. The error is at the next one.
3	**Say why**	Missing, out of order, or not clear? Name it before you touch it.
4	**Fix one thing**	Change one step. Only one. Then run it again from the beginning.

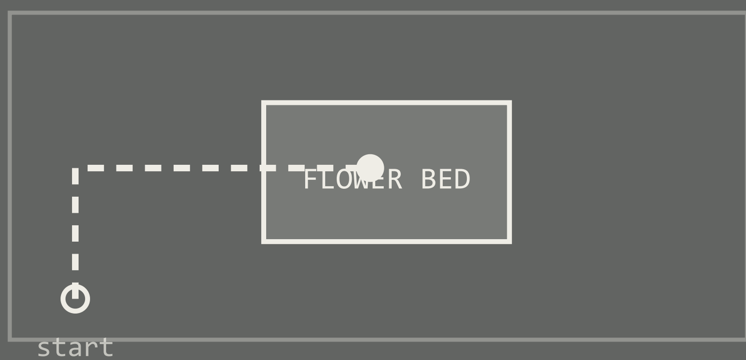
Move 4 is the one everybody breaks. If you change three steps and it works, you do not know which change fixed it, and you do not know whether the other two made something else worse.

Tracing, not guessing. Rufus keeps a claw on the line he is reading. Hazel has found something two lines below. Neither of them has picked up a pencil yet.

YOUR TURN

1 Cog was asked to walk around the flower bed and come back. Here is the plan, and here is what Cog actually did.

PLAN 1.6 · AROUND THE FLOWER BED




```
1 four paces
2 turn right
3 four paces
4 turn right
5 four paces
```

The dashed line is where Cog actually went. He stopped inside the flower bed.

a. Which step was the last one that worked? Step ____________________

b. What kind of error is it? ____________________

c. Write the step that is missing, and say where it goes.

____

It goes after step ____________________

2 Fix one thing, then run it again. Did it work?

- [ ] yes
- [ ] no

If no, do not change your fix. Go back to move 2 and find where it failed this time.

▦ **SQUARE CODE · MAZE**

A free block puzzle from Blockly Games. You build a short program to walk a character through a maze, and when it fails you can watch exactly which step went wrong. No account needed.

Why it is here. It is the fastest way to run the four debugging moves twenty times in ten minutes, which is more practice than a classroom floor allows.

AN ADULT OPENS THIS PAGE

→ **GO A BIT FURTHER**

Write a plan with a deliberate error in it, and hand it to another pair. Can they name the kind of error before they run it? Can they name it after?

Writing a bug on purpose is the best way to learn to see one by accident.

Workshop show

Build a route that another pair can walk.

You are going to write an algorithm that moves a partner from one corner of an open space to a target, without naming the target.

This is everything from this bench at once. You will write a plan, hand it to somebody who will not think for you, watch it fail, find where, and change one thing. The only new rule is that somebody else marks your work by walking it.

A ROUTE PLAN IS GOOD WHEN	A ROUTE PLAN IS WEAK WHEN
every step is a number of paces or a turn	a step says "go towards the window"
it says where to start and which way to face	it assumes the walker starts where you did
it says when to stop	it trusts the walker to notice the target
another pair can walk it in silence	the walker has to ask one question

Swapping plans. The real test of a plan is not whether you can follow it. It is whether somebody else can.

WORKSHOP SHOW · THE FOUR STEPS

1. **Mark it out.** Chalk or tape. A start square, a target square, and two obstacles in between.
2. **Write the plan.** Numbered steps, one action each. Paces and turns only. No step may name the target, and no step may say "look".
3. **Test it yourself.** Run it once with your partner as the machine. Trace it. Fix one thing at a time.
4. **Swap.** Give your plan to another pair and take theirs. Run it exactly as written, and write down where it failed and which kind of error it was.

What good looks like: they reach the target without asking you a single question.

If they do ask a question, your plan was not clear enough. That is useful information rather than a bad mark, and it tells you exactly which step to rewrite.

Write your plan here.

OUR ROUTE PLAN

1

2

3

4

5

6

DRAW THE SPACE, THE OBSTACLES AND THE ROUTE

▦ SQUARE CODE · SCRATCH

Scratch, made by the Lifelong Kindergarten group at the MIT Media Lab, free to use. Once your route plan works on the floor, you can build the same route for a character on screen.

Why it is here. It is the bridge from Unit 1 to Unit 2, where the plans you have been writing on paper become scripts.

AN ADULT OPENS THIS PAGE AND SETS UP ANY ACCOUNT

How did it go?

Colour one circle in each row: green for yes, amber for nearly, red for not yet.

AT THE PLAN BENCH	YES / NEARLY / NOT YET
My steps were in the right order	○ ○ ○
My steps were complete, with nothing missing	○ ○ ○
Each step could only be read one way	○ ○ ○
I followed my partner's plan exactly, even when I could see the mistake	○ ○ ○
I found where an error was before I changed anything	○ ○ ○
I fixed one thing at a time	○ ○ ○

☐ NOW I CAN

- ☐ say what an algorithm is, in my own words
- ☐ write a plan with steps that are in order, complete and clear
- ☐ be the machine, and follow a plan exactly as written
- ☐ explain why an error is a fault in the plan and not in the machine
- ☐ name the three kinds of error
- ☐ find where a plan first went wrong
- ☐ fix one thing, then run it again

Workshop words

Ten words. You have used every one of them at this bench.

WORD	WHAT IT MEANS
algorithm	A list of steps that solves a problem: in order, complete and clear.
step	One single action in a plan. One line, one thing to do.

WORD	WHAT IT MEANS
order	Which step comes first, next and last. Change the order and you change the result.
instruction	What one step tells you to do.
machine	Something that follows a plan without thinking about it.
error	A place where a plan does not do what its writer meant.
bug	Another word for an error, used by everybody who writes programs.
debug	To find an error and put it right.
trace	To follow a plan step by step and watch what happens at each one.
test	To run a plan on purpose, to see whether it works.